Sistemi Operativi e Gestione del Processore

Raccolta Completa: 4 Lezioni ed Esercizi

Documento consolidato con testo integrale, tabelle, formule e tutti gli schemi visivi.
Formattato appositamente per l'ingestione e lo studio su Google NotebookLM.

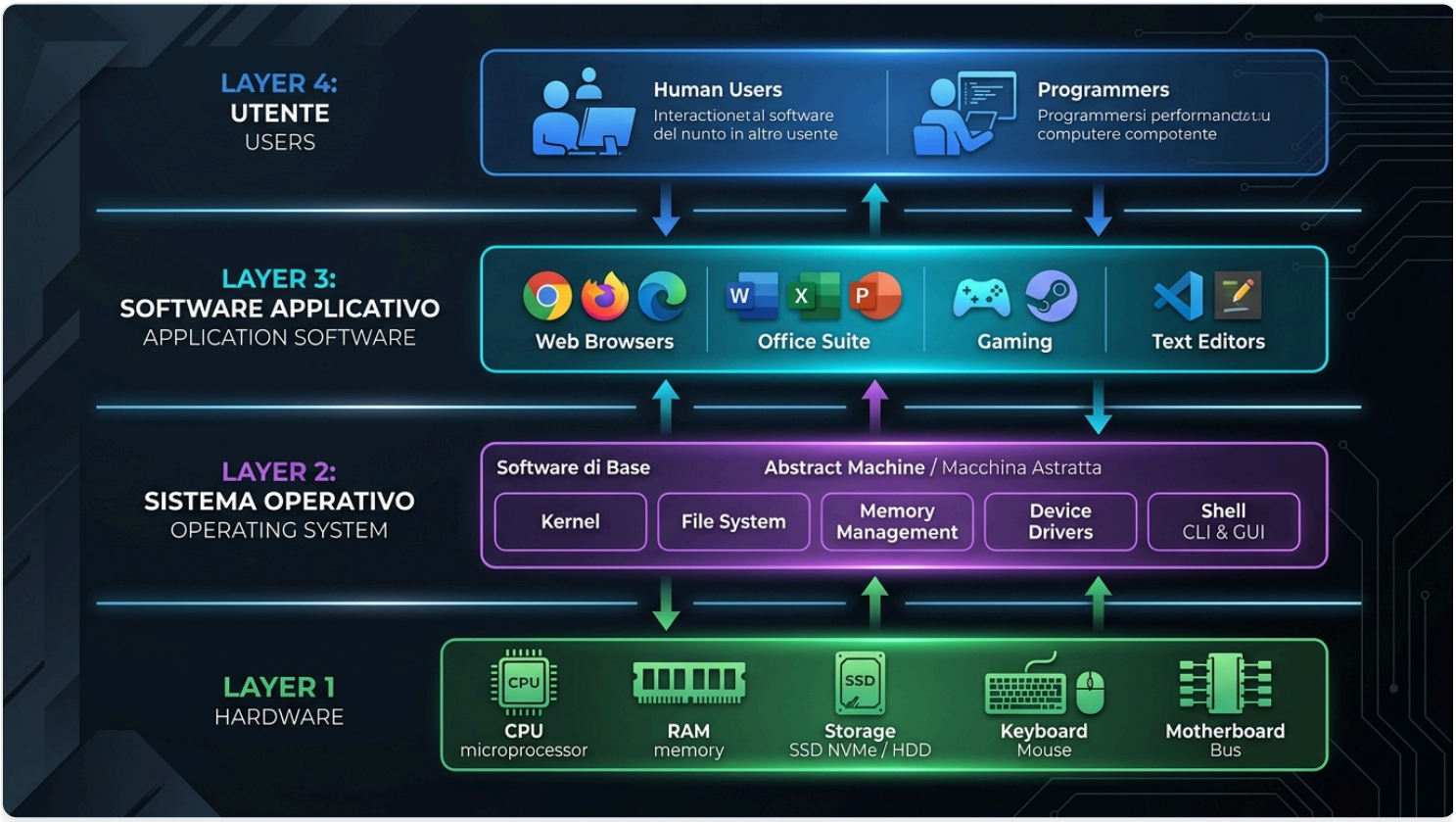
✨ Fonte Unica per Google NotebookLM

Architettura del Sistema Operativo, Boot e Privilegi

Il **Sistema Operativo (SO)** è il componente software di base più critico di un computer: senza di esso, l'hardware sarebbe un insieme inerte di silicio, piste di rame e segnali elettrici privi di coordinamento.

1.1 Il Ruolo del Sistema Operativo come Macchina Astratta e Gestore di Risorse

Il Sistema Operativo opera come un **intermediario trasparente** e insostituibile posizionato tra l'hardware fisico e i programmi applicativi utilizzati dall'utente.



> **L'Architettura a 4 Livelli del Calcolatore**

4. **Layer 4 (Utenti):** Utenti umani, programmatori e operatori di sistema.
5. **Layer 3 (Software Applicativo):** Browser web, suite d'ufficio, ambienti di sviluppo (IDE) e videogiochi.
6. **Layer 2 (Sistema Operativo - Software di Base):** Kernel, File System, Memory & Device Management, Shell CLI/GUI (la **Macchina Astratta**).
7. **Layer 1 (Hardware Fisico):** Microprocessore CPU, Memoria RAM, Dischi SSD NVMe / HDD, Periferiche I/O e Bus di sistema.

Il Concetto di Macchina Astratta (o Virtuale)

I programmi applicativi non interagiscono mai direttamente con i componenti fisici della macchina. Il SO realizza il principio di 🟡 **astrazione dell'hardware**:

- **La Macchina Fisica:** è complessa, eterogenea e vincolata a temporizzazioni hardware, indirizzi esadecimali dei registri di controllo e segnali di interrupt.
- **La Macchina Astratta:** è il modello virtuale, semplificato e standardizzato che il SO presenta ai programmatori e agli utenti. Ogni applicazione in esecuzione "crede" di avere a disposizione un computer ideale, dotato di memoria dedicata e canali di comunicazione uniformi.

> 💡 **Perché il concetto di Macchina Astratta è indispensabile?**
>
> Se non esistesse la macchina astratta, chiunque scriva un programma (ad esempio un editor di testo) dovrebbe riscrivere da zero il codice di salvataggio per **ogni specifico modello di hard disk o SSD** in commercio, conoscendo il numero esatto di piatti, cilindri, testine e settori. Grazie alla macchina astratta del SO, il programmatore invoca semplicemente l'operazione astratta `salva_file()`, demandando al SO la traduzione in comandi elettrici per il supporto di memoria secondaria. Questo garantisce la 🟢 **totale indipendenza del software dall'hardware**.

I Tre Obiettivi Fondamentali della Gestione Risorse

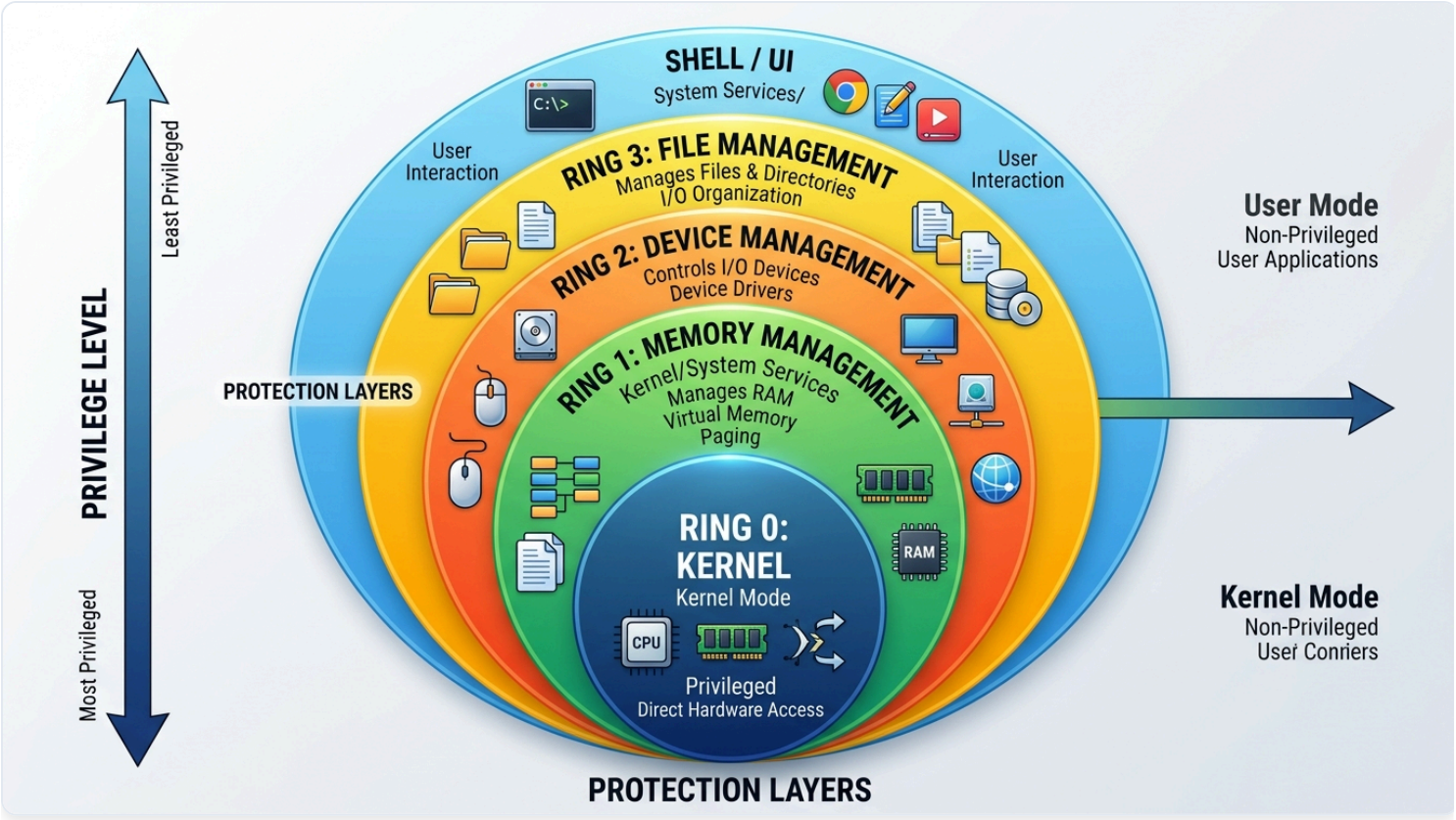
1. 🟢 **Efficienza ed Efficacia:** Ottimizzare l'allocazione delle risorse scarse (cicli CPU, blocchi RAM, banda di rete), evitando tempi morti del processore e colli di bottiglia.

2. 🟢 **Sicurezza e Affidabilità:** Isolare i processi tra loro per impedire che un'applicazione difettosa o malevola possa spiare o corrompere la memoria di altri programmi o del sistema stesso.
3. 🟢 **Trasparenza di Gestione:** Nascondere i dettagli di basso livello delle periferiche di Input/Output, permettendo all'utente di concentrarsi solo sull'attività da svolgere.

> **Classificazione Formale delle Risorse**
>
> - 🔵 **Risorse Hardware (HW):** Componenti elettronici fisici: **CPU** (tempo di elaborazione), **Memoria Centrale** (RAM, cache), **Memoria di Massa** (SSD, HDD), **Periferiche I/O** (schede di rete, monitor, tastiere) e **BUS di sistema**.
> - 🔵 **Risorse Software (SW):** Entità logiche create dal sistema: **File**, **Directory**, **Tabelle di memoria**, **Code di messaggi**, **Librerie condivise** e **Interfacce di comando**.

1.2 Architettura a Strati: Il Modello a Ring ("A Cipolla")

Per gestire la complessità e garantire la robustezza, i sistemi operativi moderni sono progettati secondo il **modello a livelli gerarchici concentrici** (*Onion Skin Architecture* o *Ring Model*):



La Gerarchia dei Livelli (Ring)🔗

Livello / Ring	Modulo Funzionale	Compiti Chiave e Gestione Sistemistica
Ring 0	🔵 **Kernel (Nucleo)**	È la componente cardine, **sempre residente in RAM** (mai soggetta a swapping su disco). Interagisce direttamente con la CPU e i registri hardware; gestisce il cambio di contesto, l'assegnazione del tempo processore e le interruzioni fisiche.
Ring 1	🔵 **Gestore della Memoria**	Alloca e dealloca le pagine di memoria centrale RAM; mappa gli indirizzi virtuali dei processi in indirizzi fisici; protegge lo spazio di memoria di ciascun processo.
Ring 2	🔵 **Gestore delle Periferiche**	Dialoga con i **driver hardware**; standardizza la comunicazione verso stampanti, schede di rete e schermi; governa i buffer di Input/Output.
Ring 3	🔵 **File System (File Manager)**	Gestisce la struttura ad albero di file e cartelle sui dischi di massa; controlla i permessi di lettura/scrittura/esecuzione e la frammentazione dei blocchi.
Esterno	🟢 **Shell (Interfaccia Utente)**	Strato più esterno che accoglie le istruzioni dell'utente, le valida sintatticamente e le converte in chiamate ai livelli interni sottostanti.

> 💡 **Perché è stata scelta un'architettura gerarchica a strati concentrici?**
>
> 1. **Principio di Minimo Privilegio e Incapsulamento:** ogni strato comunica **esclusivamente con lo strato immediatamente adiacente**. Il File System (Ring 3) non tocca direttamente i transistor della CPU o i settori elettrici del disco; chiede invece al Device Manager (Ring 2) di trasmettere i byte.
> 2. **Modularità e Manutenibilità:** se si sostituisce un componente hardware (ad esempio si installa una nuova scheda video o si passa da un disco SATA a un SSD NVMe), è sufficiente aggiornare il driver nel Ring 2. Tutti i livelli superiori (File System, Shell e Programmi utente) rimangono identici e continuano a funzionare senza dover essere riscritti.

Dettaglio della Shell: Le 3 Tipologie di Interfaccia Utente

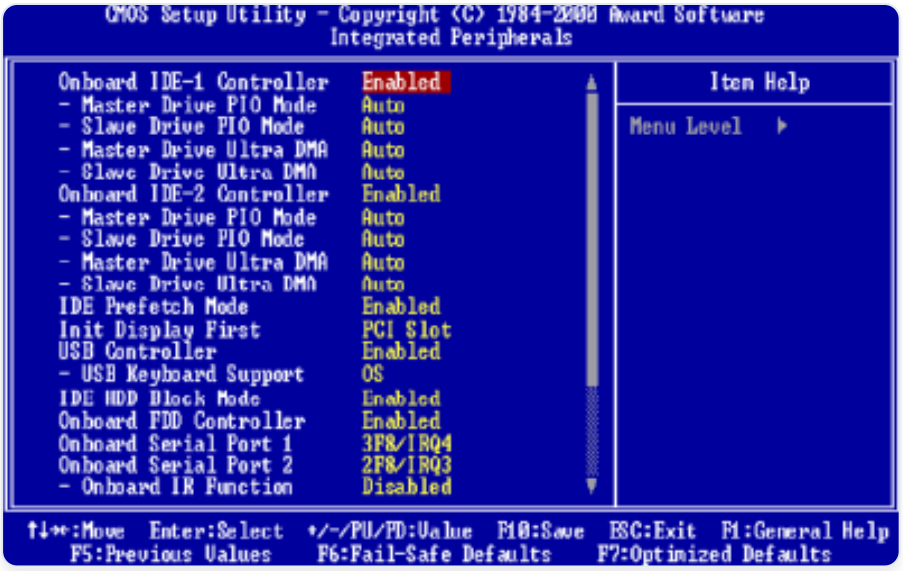
1. **CLI (Command Line Interface):** Interfaccia a riga di comando testuale. L'utente digita comandi che vengono analizzati da un interprete sintattico (**parser**). *Esempi:* Bash su Linux/macOS, `cmd.exe` e PowerShell su Windows. È preferita da amministratori di sistema per la sua leggerezza, rapidità e automazione tramite script.

   Transclude of public.avif

2. **GUI (Graphical User Interface):** Interfaccia grafica intuitiva basata sulla **metafora della scrivania (desktop)** e sul paradigma **WIMP** (*Windows, Icons, Menus, Pointer*). *Esempi:* Windows Explorer, macOS Aqua, GNOME/KDE su Linux. Offre massima immediatezza visiva a fronte di un maggior consumo di RAM e CPU.

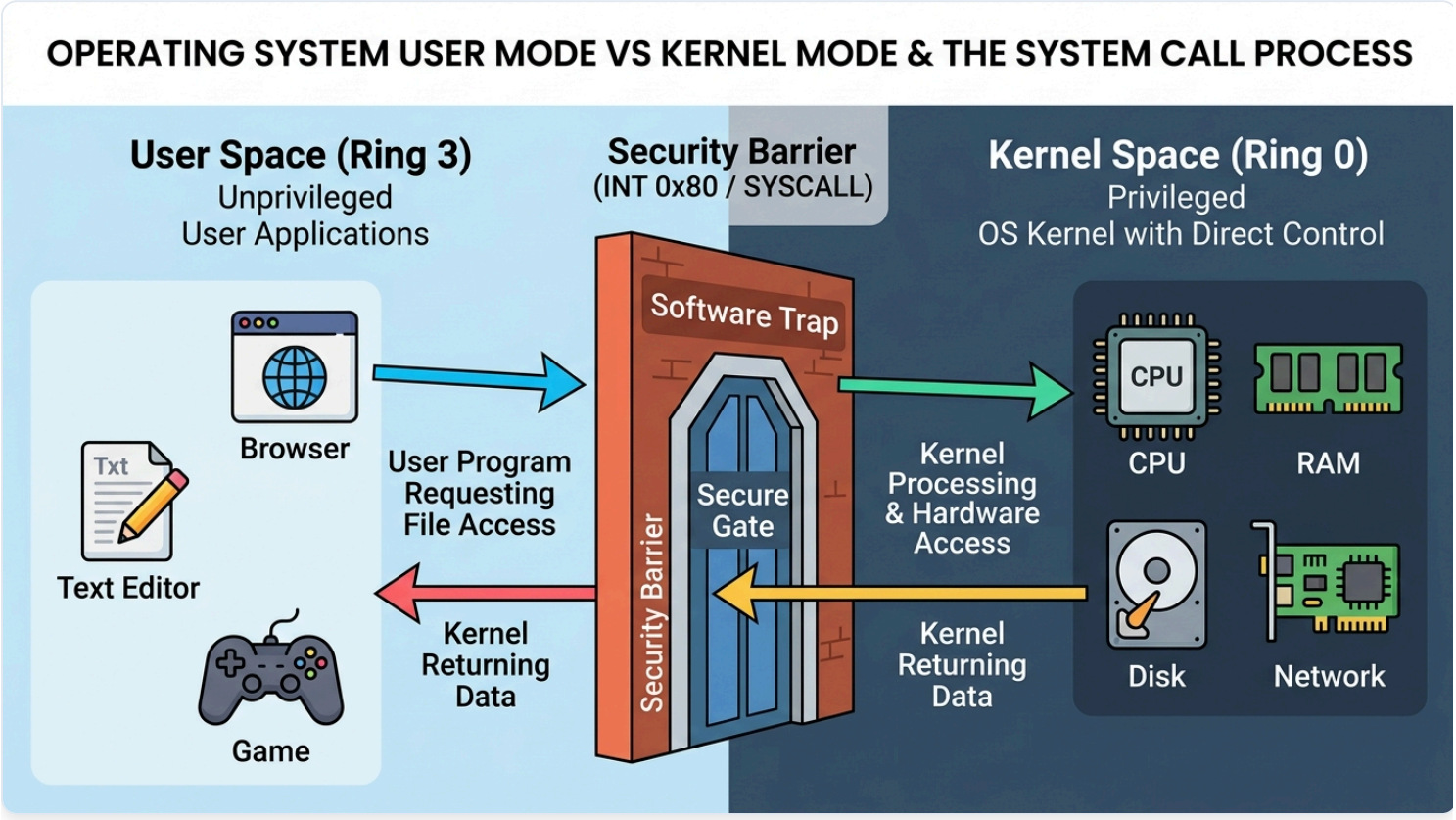


3. **Menu-Driven Interface:** Interfaccia basata sulla selezione da menu testuali gerarchici. Non richiede digitazione libera di comandi complessi ed è utilizzata negli ambienti a basso livello privi di driver grafici avanzati (ad esempio le schermate di configurazione del **BIOS/UEFI**).



1.3 Livelli di Privilegio: User Mode, Kernel Mode e System Call

Per evitare che un errore logico (un ciclo infinito o un puntatore nullo) o un'azione malevola di un programma utente possano bloccare l'intero computer, i processori integrano una **distinzione hardware degli stati di privilegio**:



Confronto Diretto: User Mode vs Kernel Mode

Caratteristica	User Mode (Modalità Utente)	Kernel Mode (Modalità Protetta / Ring 0)
Livello di Privilegio	🟢 **Basso / Limitato** (non privilegiato).	🔴 **Massimo / Totale** (modalità supervisore).
Componenti Eseguite	Applicazioni utente ordinarie, script, shell.	Nucleo del SO (Kernel), driver di sistema, scheduler.
Accesso alla Memoria	Limitato allo spazio di indirizzamento virtuale proprio del processo.	Diretto e illimitato all'intera memoria RAM fisica.
Istruzioni CPU Permesse	Solo istruzioni aritmetico-logiche ordinarie.	Qualsiasi istruzione macchina, incluse quelle privilegiate.
Conseguenza di un Errore	🟢 **Chiusura forzata della singola applicazione** (Crash dell'app, nessun impatto sul sistema).	🔴 **Arresto critico dell'intero calcolatore** (BSOD su Windows, Kernel Panic su Linux).

Il Meccanismo della System Call e la Trap Software

Quando un programma in User Mode necessita di accedere a una risorsa hardware (es. leggere un file, allocare memoria, connettersi a un socket di rete), non può farlo direttamente. Deve invocare una 🟣 **System Call (Chiamata di Sistema)**:

1. L'applicazione prepara i parametri nei registri della CPU o sullo stack.
2. Viene eseguita un'istruzione di 🟣 **Trap Software / Interruzione Software** (ad esempio `INT 0x80` nell'architettura x86 a 32 bit o l'istruzione speciale `SYSCALL` / `SYSENTER` a 64 bit).
3. **Cosa fa l'Hardware:** La CPU intercetta la trap, sospende il programma utente, commuta il bit di privilegio da User Mode (Ring 3) a Kernel Mode (Ring 0) e salta all'indirizzo del vettore delle chiamate di sistema del Kernel (*System Call Table*).
4. Il Kernel esegue i controlli di sicurezza (es. "l'utente ha i permessi di lettura sul file?"), compie l'operazione hardware e rimette il risultato nei registri.
5. Con l'istruzione di ritorno (`IRET` o `SYSRET`), la CPU ripristina la modalità User Mode e riconsegna il controllo al programma chiamante.

> 💡 **Perché è la CPU e non il programma a cambiare la modalità in Kernel Mode?**

> Se un programma utente potesse cambiare autonomamente la modalità della CPU da User Mode a Kernel Mode mediante una semplice istruzione software ordinaria, **qualsiasi virus o malware potrebbe proclamarsi amministratore assoluto del computer in microsecondi**. La commutazione può avvenire esclusivamente mediante una **trappola hardware**: è la circuiteria interna del microprocessore che riconosce l'evento eccezionale e forza l'accesso solo attraverso punti di ingresso rigidamente controllati e validati dal Kernel.

> **Esecuzione Non-Preemptive in Kernel Mode**
>
> Le routine interne del Kernel (ad esempio il salvataggio dei registri della CPU durante il cambio di contesto o l'aggiornamento della tabella delle pagine di memoria) devono essere **rigorosamente non-preemptive (non interrompibili)**:
>
> - Se un'interruzione esterna bloccasse la CPU proprio mentre sta modificando i puntatori della lista dei processi o i registri di stato, il sistema si ritroverebbe in uno 🔴 **stato incoerente e corrotto**, provocando il crash istantaneo dell'intera macchina.

1.4 Sequenza di Avvio del Sistema Operativo (Bootstrap) e Ambienti Isolati

All'atto dell'accensione del calcolatore, la memoria centrale RAM è completamente vuota (volatile). Il caricamento del Sistema Operativo è un processo automatico suddiviso in **5 fasi sequenziali** noto come **Bootstrap**:

Le 5 Fasi Dettagliate del Bootstrap

1. 🔵 **Accensione e Attivazione del Firmware (Reset Vector):** La corrente attiva la CPU. Il registro Program Counter della CPU viene pre-impostato via hardware su un indirizzo cablato noto come **Reset Vector** (es. `0xFFFFFFF0` su architetture x86 o `0x00000000` su ARM-64), dove si trova la prima istruzione del firmware permanente in ROM/NVRAM (**BIOS** o **UEFI**).
2. 🔵 **Esecuzione del POST (Power-On Self-Test):** Il firmware esegue un test diagnostico dei componenti fisici fondamentali (presenza e integrità di CPU, moduli RAM, scheda video, bus

e controller). Se si riscontra un guasto critico, la scheda madre emette sequenze sonore (*Beep Codes*) o codici numerici esadecimali su display integrato.

3. 🔵 **Inizializzazione Hardware e Lettura Partizioni:** Vengono configurati i canali DMA, i canali PCIe e i clock. In base alla sequenza di avvio impostata (*Boot Order*), il firmware interroga il disco di avvio e legge la tabella delle partizioni (**MBR** o **GPT**).
4. 🔵 **Caricamento del Bootloader:** Il firmware individua il settore o la partizione di avvio e copia nella RAM il **Bootloader** (es. *GRUB2* in Linux, *Windows Boot Manager*, *rEFInd*). Se sono presenti più sistemi operativi, il Bootloader mostra il menu di scelta (*Boot Menu*).
5. 🔵 **Caricamento del Kernel e Inizializzazione Utente:** Il Bootloader carica l'eseguibile del Kernel nello spazio di memoria riservato della RAM (*Kernel Space*), cede il controllo della CPU al processo padre primario (denominato `init` o `systemd` in ambiente Linux, `smss.exe` in ambiente Windows) che avvia i demoni di servizio in background e, infine, rende disponibile la Shell grafica o testuale.

⚙️ Modalità di Elaborazione ed Entità Software

La progettazione dei sistemi operativi si è evoluta per risolvere un problema storico dell'informatica: **la colossale discrepanza di velocità tra la CPU e i dispositivi di Input/Output**. Da qui derivano le diverse modalità con cui l'elaboratore gestisce i programmi e alloca il tempo di calcolo.

2.1 Le Modalità di Elaborazione del Sistema Operativo

Nel corso dell'evoluzione tecnologica, i sistemi operativi sono stati classificati in base al modo in cui accolgono le richieste e ripartiscono le risorse hardware:

> 🖥️ **Le 4 Modalità Fondamentali di Elaborazione del SO**
>
> 1. 1️⃣ **BATCH (A Lotti):** Esecuzione sequenziale a pacchetti senza interazione utente (es. calcolo buste paga, backup notturni).
> 2. 2️⃣ **INTERATTIVA:** Dialogo bidirezionale in tempo reale uomo-macchina (es. browser web, suite office, sportelli ATM).
> 3. 3️⃣ **MULTIPROGRAMMAZIONE & TIME-SHARING:** Più processi residenti in RAM, alternati da quanti di tempo (*Time-Slice*) e abilitati dal controller DMA.
> 4. 4️⃣ **REAL-TIME (Tempo Reale):** Correttezza vincolata al rispetto categorico di scadenze temporali certe (*Deadlines* - Hard vs Soft).

---

1. Sistemi Batch (A Lotti)

- **Funzionamento:** I programmi da eseguire (**job**) vengono raggruppati in un "lotto" (*batch*) ed eseguiti dalla CPU in modo sequenziale, uno dopo l'altro.
- **Interazione Utente:** 🔴 **Completamente assente** durante la fase di calcolo. Tutti i dati di ingresso (*input*) devono essere noti e caricati all'inizio. Se il programma contiene un errore

logico, l'elaborazione fallisce e l'utente ne viene a conoscenza solo al termine dell'intero blocco.
- **Utilizzo Tipico:** Operazioni ad alta intensità di calcolo che non richiedono supervisione umana (es. elaborazione dei cedolini delle buste paga a fine mese nei centri di calcolo aziendali, rendering 3D offline, backup notturni di database).

---

2. Sistemi Interattivi

- **Funzionamento:** È presente un dialogo continuo e bidirezionale tra l'utente e il sistema tramite periferiche di I/O (tastiera, mouse, schermo).
- **Tempo di Risposta:** Il sistema deve garantire tempi di risposta rapidi (frazioni di secondo o pochi secondi), affinché l'utente percepisca una risposta immediata a ogni comando digitato o clic eseguito.
- **Utilizzo Tipico:** Sistemi operativi consumer (Windows, macOS, smartphone, sportelli bancomat ATM, browser web).

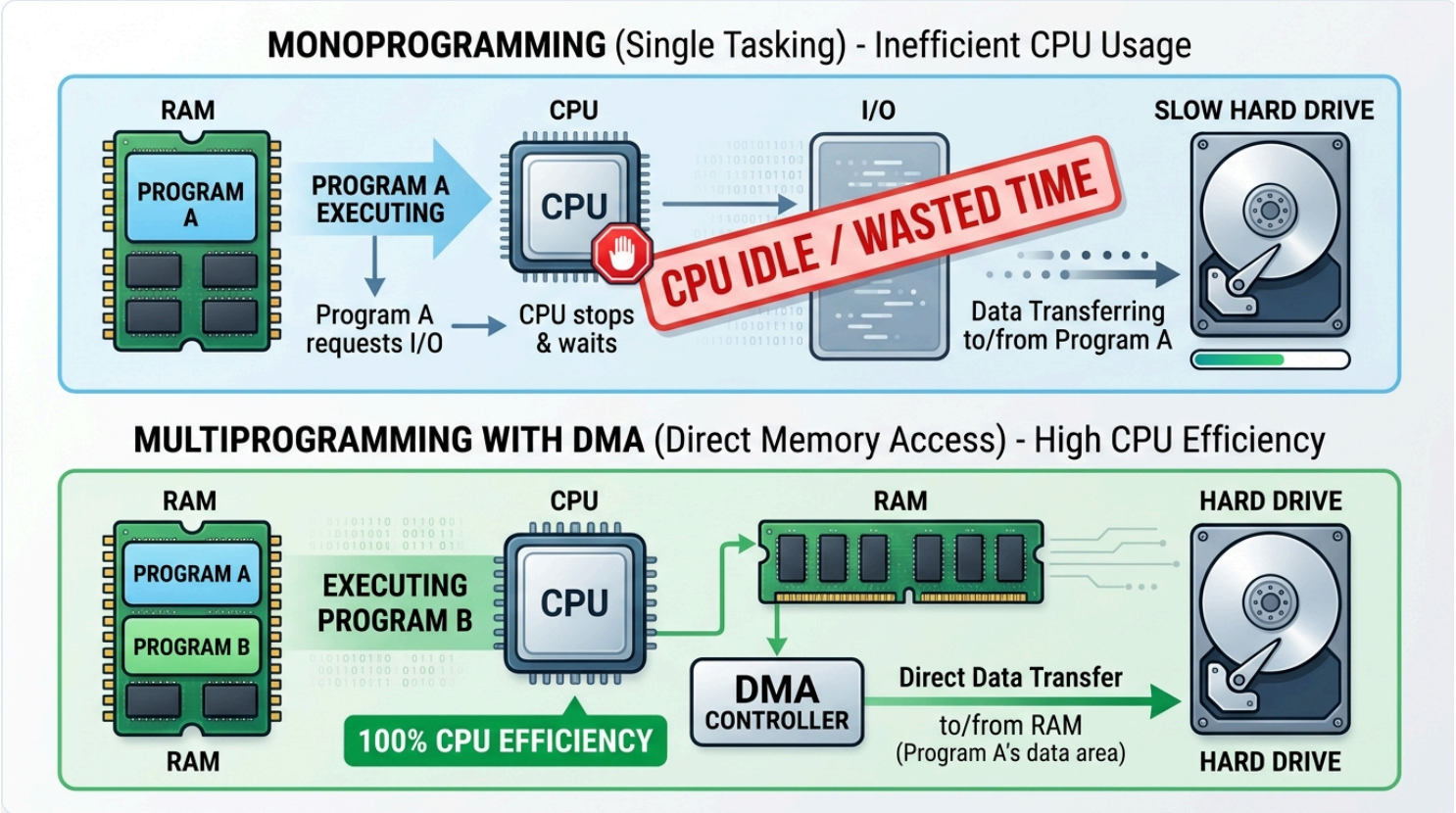
---

3. Dalla Monoprogrammazione alla Multiprogrammazione e Multitasking

Per comprendere la nascita della multiprogrammazione, occorre analizzare la **differenza di velocità tra i componenti hardware**:

> ⏱️ **La Discrepanza di Velocità Hardware (L'Origine dell'I/O Bottleneck)**

Componente Hardware	Ordine di Grandezza Temporale	Rallentamento Rispetto alla CPU
⚡ **Ciclo di Clock della CPU**	≈ 0,3 nanosecondi (10−9 s)	**Unità di riferimento (Velocità Massima)**
🧠 **Accesso alla Memoria RAM**	≈ 10 – 50 nanosecondi	≈ 30 × −150× più lento
💽 **Lettura SSD NVMe veloce**	≈ 10 – 50 microsecondi (10−6 s)	≈ 30.000 × −150.000× più lento
💿 **Lettura da Hard Disk Meccanico (HDD)**	≈ 5 – 10 millisecondi (10−3 s)	🔴 **Oltre 30 MILIONI di volte più lento!**
⌨️ **Digitazione Umana su Tastiera**	≈ 0,2 – 1 secondo	🔴 **MILIARDI di volte più lento!**



- **Monoprogrammazione:** In RAM risiede **un solo programma utente** alla volta insieme al Kernel. Quando il programma richiede la lettura di un dato da disco o l'attesa di un tasto dalla tastiera, la CPU si arresta completamente (*busy waiting*), rimanendo inattiva per milioni di cicli di clock.
- **Multiprogrammazione:** In RAM sono allocati **più programmi contemporaneamente**.
    - Sfrutta i canali di **Direct Memory Access (DMA)**: un circuito intelligente dedicato che trasferisce blocchi di dati tra le periferiche e la RAM **senza gravare sulla CPU**.
    - Quando il processo corrente richiede un'operazione di I/O, il SO sospende quel processo e assegna istantaneamente la CPU a un altro processo già pronto in RAM.

> 💡 **Perché la Multiprogrammazione ha rivoluzionato l'informatica?**
>
> Ha sconfitto il devastante fenomeno dell' 🔴 **I/O Bottleneck (Collo di Bottiglia dell'I/O)** . In un sistema monoprogrammato, la CPU più potente del pianeta rimarrebbe inattiva per il **95-99% del tempo** in attesa dei dischi o dell'utente. Con la multiprogrammazione e il DMA, la percentuale di utilizzo utile della CPU (**throughput ed efficienza η**) sale vertiginosamente verso livelli prossimi al 100%.

---

4. Sistemi Time-Sharing (A Divisione di Tempo)

I sistemi **Time-Sharing** rappresentano l'evoluzione logica della multiprogrammazione applicata agli ambienti interattivi multiutente:

- Il tempo totale della CPU viene suddiviso in piccoli intervalli temporali prefissati chiamati 🔴 **Quanti di Tempo o Time-Slice** (tipicamente compresi tra 10 e 100 millisecondi).
- La CPU viene assegnata a rotazione circolare a ciascun processo pronto in memoria per la durata massima di un time-slice.
- Grazie all'elevata frequenza di commutazione, ciascun utente ha l'illusione ottica e operativa di possedere una CPU dedicata esclusivamente a sé.

> **I 3 Eventi Esatti di Rilascio Forzato della CPU**
>
> Durante l'elaborazione in un sistema Time-Sharing, un processo in esecuzione nella CPU è obbligato a **rilasciare immediatamente il processore** al verificarsi di uno solo tra questi **3**

eventi esatti:

1. 🟢 **Il programma termina regolarmente la propria esecuzione** (raggiunge l'istruzione di fine programma e transita nello stato *TERMINATED*).
2. 🟣 **Il programma richiede un'operazione di I/O o attende un evento esterno** (es. attesa di un pacchetto di rete o lettura disco; transita nello stato *WAITING*).
3. 🔴 **Scade il Time-Slice assegnato**: il timer hardware della scheda madre genera un'interruzione per timeout; il processo viene prelazionato forzatamente e rimesso nella coda dei pronti (*READY*).

Confronto: Windows Desktop vs Linux Multiutente

- **Windows Desktop:** È un sistema time-sharing 🟡 **monoutente**. Consente di configurare più account utente sulla macchina, ma permette a una sola sessione utente interattiva di essere attiva e controllabile alla console fisica per volta.
- **Linux:** È un sistema time-sharing 🟢 **multiutente nativo**. Più utenti umani distinti possono essere collegati contemporaneamente alla stessa macchina (ad esempio tramite connessioni di rete cifrate **SSH** o terminali virtuali multipli `tty1..tty6`), eseguendo ciascuno i propri programmi in parallelo con memoria e permessi rigorosamente isolati.

---

5. Il Compromesso Vitale: System Overhead e Scelta del Time-Slice

Definizione di System Overhead (Sovraccarico di Sistema)

Il **System Overhead** è la porzione di tempo e di cicli CPU che il Sistema Operativo consuma **per le proprie attività burocratiche interne di gestione** (in particolare l'esecuzione del *Context Switch*, ovvero il salvataggio dei registri nel PCB e la selezione del nuovo processo), anziché dedicarla al calcolo utile per i programmi degli utenti.

⚖️ Analisi Comparativa: Impatto della Dimensione del Time-Slice (TQ)

Dimensione Time-Slice	Comportamento Interno del Sistema	Impatto sull'Efficienza e sull'Esperienza Utente
🔴 **Troppo Piccolo** (es. 1 ms)	Frequenti Context Switch continui. La CPU passa gran parte del proprio tempo a salvare e ripristinare i PCB invece di eseguire calcoli.	🔴 **L'Efficienza (η) crolla** drammaticamente a causa dell'enorme overhead burocratico.
🔴 **Troppo Grande** (es. 1000 ms)	Il sistema degenera in un algoritmo sequenziale FCFS. I processi lunghi monopolizzano la CPU.	🔴 **L'interfaccia grafica si blocca** e l'utente percepisce forti rallentamenti e lag nei comandi.
🟢 **Ottimale** (10 – 50 ms)	Il tempo del Context Switch incide per meno dell'1% del ciclo complessivo.	🟢 **Compromesso perfetto**: massima reattività dell'interfaccia grafica e minima dispersione di CPU.

💡 Qual è il ragionamento matematico per scegliere la grandezza del Time-Slice?

Sia TQ la durata del time-slice e TCS il tempo hardware necessario a compiere un Context Switch (tipicamente 5 – 10 microsecondi).

- Se impostassimo un time-slice minuscolo, ad esempio TQ = 20 µs, avremmo un cambio di contesto ogni 20 µs. La CPU spenderebbe un terzo del suo tempo solo a gestire i PCB!
- Per convenzione ingegneristica, il time-slice TQ deve essere dimensionato in modo che il tempo di cambio di contesto rappresenti **meno dell'1% del tempo totale**: $\frac{TCS}{TQ} \leq 0{,}01 \implies TQ \geq 100 \times TCS$ Un intervallo di 10 – 50 ms garantisce che l'overhead sia trascurabile senza degradare la reattività dell'interfaccia grafica.

---

6. Sistemi Real-Time (A Tempo Reale)

Nei sistemi **Real-Time**, la correttezza del sistema non dipende solo dal risultato logico del calcolo, ma **dal momento esatto in cui il risultato viene prodotto**:

- L'avanzamento dei calcoli è scandito da **eventi fisici esterni** (sensori, sonde, attuatori).
- I programmi critici risiedono **sempre in RAM** (mai soggetti a memoria virtuale o swapping).
- Ogni task ha una scadenza temporale prefissata e categorica chiamata 🔴 **Deadline**.

Caratteristica	Hard Real-Time (Vincolo Stretto)	Soft Real-Time (Vincolo Largo)
Rigidità della Scadenza	🔴 **Assoluta e Invalicabile.**	Più elastica / Tollerante a piccoli ritardi.
Effetto del Mancato Rispetto	**Dato corrotto / Fallimento totale catastrofico:** pericolo per la vita umana o distruzione dell'impianto.	**Degrado del Servizio (SLA / QoS):** rallentamento momentaneo o perdita di qualità percepita.
Sistemi Operativi Esempio	QNX Neutrino, VxWorks, FreeRTOS.	Kernel Linux con patch standard, Windows Media Player.
Ambiti di Applicazione Reali	• Apertura degli **Airbag** nell'auto (entro 20-30 ms dall'urto). • Sistemi di controllo barre di controllo in **Centrali Nucleari**. • Controllo di volo avionico *Fly-by-Wire* e pacemaker cardiaci.	• **Streaming video/audio** (se un frame salta, si avverte un micro-scatto). • Sistemi di prenotazione biglietti aerei online. • Videogiochi multiplayer online.

2.2 Evoluzione delle Entità Software: Programma, Job, Processo e Thread

Uno degli aspetti più formativi della teoria dei sistemi operativi è comprendere come un file memorizzato sul disco si trasforma in un'applicazione attiva in esecuzione:

> 🔄 **La Pipeline di Trasformazione delle Entità Software**

1. 💾 **Programma su Disco:** File eseguibile binario memorizzato su memoria secondaria (inattivo, 0% CPU, 0% RAM).
2. ⏳ **Job in HOLD (JCB su Disco):** Richiesta di avvio (doppio clic) parcheggiata in attesa che il Long-Term Scheduler verifichi le risorse.
3. 🧠 **Processo in RAM (PCB in Memoria):** Istanza attiva e dinamica in memoria protetta, pronta a concorrere per la CPU.
4. 🧵 **Thread (Flussi di Esecuzione Leggeri):** Più rami di calcolo paralleli interni al processo che condividono memoria Heap e codice.

Le 4 Entità a Confronto

1. 🔵 **Programma:** Entità **statica e passiva**. È il file eseguibile memorizzato permanentemente su memoria di massa (disco SSD/HDD, es. `chrome.exe`). È costituito da codice binario, istruzioni macchina e dati iniziali. **Non consuma cicli CPU né celle di memoria RAM** finché non viene invocato.
2. 🔵 **Job (Lavoro):** Entità **transitoria e preliminare**. Si genera nel momento esatto in cui si richiede l'avvio del programma (es. al doppio clic dell'icona). Il programma viene posto in una lista di attesa su disco denominata stato 🟡 **NEW / HOLD**. Durante questa fase il SO valuta se ci sono abbastanza risorse hardware per ammetterlo.
3. 🟢 **Processo (Task):** Entità **dinamica e attiva**. È l'istanza concreta di un programma caricato nella memoria centrale **RAM**, a cui il SO ha assegnato uno spazio di memoria protetto (*Address Space*), descrittori di file e identificatori univoci. È il processo che concorre attivamente per ottenere il tempo di calcolo della CPU.
4. 🟢 **Thread ("Filo di Esecuzione"):** Unità di esecuzione **leggera** (*Lightweight Process*) interna a un processo. Più thread appartenenti al medesimo processo **condividono lo stesso spazio di memoria RAM e i file aperti del processo padre**, ma ciascuno possiede il proprio **Program Counter (PC)**, il proprio set di registri della CPU e il proprio stack di chiamate.

💡 Perché sono stati inventati i Thread al posto di creare tanti Processi?

1. **Economia di Memoria:** Creare un processo richiede al SO di duplicare l'intera tabella delle pagine di memoria virtuale e allocare nuove strutture dati complesse (pesantezza in termini di megabyte). Creare un thread richiede solo poche centinaia di byte per registri e stack.
2. **Velocità di Comunicazione (Inter-Process Communication):** Due processi distinti sono isolati dalla memoria e per comunicare devono fare lente chiamate di sistema

(*pipe, socket, shared memory*). Due thread dello stesso processo leggono e scrivono sulle **stesse identiche variabili globali in memoria RAM** a velocità nativa.
3. **Context Switch Immediato:** Passare la CPU da un thread all'altro dello stesso processo non richiede lo svuotamento della cache della memoria di traduzione (**TLB** - *Translation Lookaside Buffer*), rendendo il cambio di contesto istantaneo.

2.3 Strutture Dati di Controllo: JCB vs PCB

Per governare l'evoluzione di ciascun Job e Processo, il Sistema Operativo crea e mantiene aggiornate due strutture dati fondamentali in memoria:

Parametro / Campo	JCB (Job Control Block)	PCB (Process Control Block)
Momento di Creazione	**All'avvio del Job**, mentre il programma si trova nello stato *NEW/HOLD* su disco.	**All'atto dell'ammissione** del processo nella memoria centrale RAM (stato *READY*).
Supporto di Residenza	Memorizzato nella **Lista di HOLD su disco**.	Memorizzato nello spazio protetto del **Kernel in RAM**.
Identificativo	**Job ID** (identificativo del lavoro in coda).	🔵 **PID (Process Identifier)**: numero intero univoco assegnato al processo.
Dati Identificativi e Spazio	• Percorso del file eseguibile su disco. • Dimensione stimata del codice in KB. • Elenco delle periferiche I/O richieste.	• Puntatore alla base del codice in memoria RAM. • Tabella delle pagine di memoria virtuale. • Descrittori dei **file attualmente aperti** (*Open File Table*).
Contesto della CPU	*Non presente* (il job non ha ancora mai toccato la CPU).	🔵 **Stato completo dei Registri CPU**: accumulatori, registri generali, registro di stato (**PSW** - *Program Status Word*) e **Program Counter (PC)**.
Stato e Metriche di Tempo	• Priorità di ammissione. • Tempo massimo di esecuzione consentito.	• **Stato corrente** (*Ready, Running, Waiting...*). • Priorità di scheduling. • Istante di arrivo (TA), tempo di burst consumato (TB) e tempo di attesa cumulata.
Organizzazione Strutturale	Elenco sequenziale nella coda dei lavori su disco.	**Organizzato in liste dinamiche e code circolari concatenate in RAM** (ogni PCB contiene un puntatore `next` al PCB successivo).

> 💡 **Perché il JCB è gestito su disco mentre il PCB deve risiedere rigorosamente in RAM?**
>
> Nello stato preliminare di HOLD possono essere accodati centinaia di programmi in attesa (ad esempio in un grande server universitario o aziendale). Se creassimo subito i PCB in memoria RAM per tutti i programmi proposti, **esauriremmo immediatamente la memoria centrale**, portando il sistema in blocco prima ancora di iniziare (*Thrashing*). Il JCB risiede su disco per fare da "sala d'attesa a basso costo"; solo quando il *Job Scheduler* accerta che c'è abbastanza RAM libera, crea il PCB fisico in memoria centrale.

🔄 Ciclo di Vita del Processo e Algoritmi di Scheduling

La CPU è il “motore” del calcolatore: in un’architettura monoprocessore (o all’interno di un singolo core), **un solo processo alla volta può trovarsi in esecuzione fisica**. Il coordinamento di tutti gli altri processi presenti nel computer spetta ai moduli specializzati del Kernel e agli **algoritmi di scheduling**.

3.1 Il Diagramma degli Stati del Processo (Ciclo di Vita)

Durante la propria esistenza, un programma non passa istantaneamente dal disco all’esecuzione, ma attraversa una serie di **stati logici** codificati nel campo `Stato` del proprio **PCB (Process Control Block)**.

> 🧭 **Guida alla Lettura del Diagramma**
>
> Il diagramma modella l’intero ciclo di vita del processo tramite **5 stati fondamentali** (le ellissi), **6 transizioni orientate** (le frecce) e **2 code dinamiche in memoria RAM** (**RL** e **WL**).

I 5 Stati Fondamentali del Processo

Stato	Denominazione nel Diagramma	Collocazione Fisica	Condizione Operativa del Processo
🟡 Nuovo	**Nuovo (New)**	Memoria di Massa (Disco)	Il programma è stato appena invocato dall'utente (doppio clic o comando da terminale). Esiste la richiesta di avvio, ma il processo non è ancora stato caricato in RAM.
🔵 Pronto	**Pronto (Ready)**	Memoria Centrale (RAM)	Il processo è completamente caricato in memoria RAM con tutte le risorse necessarie: **è pronto a calcolare e attende solo che la CPU gli venga assegnata**. Risiede nella **RL (Ready List)**.
🟢 In esecuzione	**In esecuzione (Running)**	Microprocessore (CPU)	Il processo ha ottenuto il controllo della CPU e ne sta eseguendo le istruzioni nei registri interni. In un sistema monoprocessore (o singolo core), **un solo processo alla volta** può trovarsi in questo stato.
🔴 Sospeso	**Sospeso (Waiting / Blocked)**	Memoria Centrale (RAM)	Il processo non può proseguire l'esecuzione perché **è in attesa del completamento di un'operazione di I/O** (disco, tastiera, rete) o di un evento esterno (timer). Risiede nella **WL (Waiting List)**.
🟡 Finito	**Finito (Terminated)**	Rilascio Risorse	Il processo ha concluso regolarmente il suo compito (istruzione `exit()`) oppure è stato terminato forzatamente per un errore irreversibile. Il PCB viene distrutto e la memoria RAM liberata.

> 💡 **Qual è la differenza fondamentale tra lo stato "Pronto" e "Sospeso"? (Domanda della Prof!)**
>
> - Il processo in 🔵 **Pronto (Ready)** possiede già tutto il necessario (codice, dati e memoria). Gli manca **soltanto la disponibilità della CPU**. Se la CPU si liberasse in questo istante, potrebbe iniziare a calcolare immediatamente.
> - Il processo in 🟠 **Sospeso (Waiting)** è bloccato da una dipendenza esterna: anche se la CPU fosse completamente libera al 100%, **non potrebbe comunque eseguire alcuna istruzione**, poiché deve prima ricevere i dati dalla periferica di I/O o attendere l'evento asincrono.

Le 6 Transizioni di Stato Spiegate Passo-Passo

Le frecce orientate del diagramma rappresentano i passaggi di stato governati dal Kernel:

1. 🔵 **Ammissione ($\text{Nuovo} \longrightarrow \text{Pronto}$):**
   - **Cosa accade:** Il sistema operativo ammette la nuova richiesta di esecuzione, crea il descrittore PCB in memoria RAM e inserisce il processo nella coda dei pronti (**RL**).
2. 🟢 **Dispatching / Assegnazione CPU ($\text{Pronto} \longrightarrow \text{In esecuzione}$):**
   - **Cosa accade:** La CPU diventa libera. Il modulo **Dispatcher**, guidato dalla politica dello scheduler, preleva il processo in testa alla **RL**, esegue il *Context Switch* caricando i suoi registri nella CPU e commuta l'hardware in User Mode avviando il calcolo.
3. 🟣 **Interrotto / Revoca CPU ($\text{In esecuzione} \longrightarrow \text{Pronto}$):**
   - **Cosa accade:** La CPU viene revocata forzatamente al processo per **prelazione (preemption)**:
     - Nei sistemi *Time-Sharing*, è scaduto il quanto di tempo (**Time-Slice**) concesso dal timer di sistema.
     - Nei sistemi con priorità, è arrivato nella RL un processo con priorità più alta.
     - Il processo sospeso salva il proprio stato nel PCB e torna in coda nella **RL**.
4. 🟠 **Sospensione ($\text{In esecuzione} \longrightarrow \text{Sospeso}$):**
   - **Cosa accade:** Il processo in esecuzione richiede un'operazione lenta di I/O (es. lettura da disco o input utente). Non potendo procedere, **cede spontaneamente la CPU** (tramite una System Call) per non sprecare cicli di calcolo a vuoto e si accoda nella **WL**.
5. 🟢 **Riattivazione ($\text{Sospeso} \longrightarrow \text{Pronto}$):**

- **Cosa accade:** La periferica termina l'operazione di I/O e invia un interrupt hardware (**IRQ**). L'*Interrupt Handler* prende in carico i dati e sposta il processo dalla WL alla **RL**. *(Regola d'oro: il processo torna in stato Pronto, mai direttamente in esecuzione sulla CPU!)*

6. 🔴 **Terminazione ($\text{In esecuzione} \longrightarrow \text{Finito}$):**
   - **Cosa accade:** Il processo completa la sua ultima istruzione o subisce un errore irreversibile. Il sistema operativo distrugge il PCB, chiude i file aperti e restituisce la memoria RAM al sistema.

Le Due Code in Memoria: RL e WL

Nel diagramma sono disegnate due strutture dati a colonna che affiancano gli stati in RAM:

- 🔵 **RL (Ready List):** La coda dinamica dei processi in stato *Pronto*, ordinati secondo la logica dell'algoritmo di scheduling adottato (es. ordine FIFO, per tempo residuo o per priorità decrescente).
- 🔴 **WL (Waiting List):** L'insieme delle code di attesa dei processi in stato *Sospeso*. Nei sistemi reali non esiste un'unica lista generica, ma **una coda separata per ciascun dispositivo di I/O** (coda della tastiera, coda del disco, coda della scheda di rete), garantendo che ogni periferica risvegli esclusivamente i processi che attendono il suo specifico segnale.

🛠️ Il Ruolo dei 6 Moduli del Kernel e la Loro Architettura

1. Moduli di Ammissione e Gestione Memoria (Disco ↔ RAM)

> 📁 **1. Long-Term Scheduler (Job Scheduler / LTS)**
>
> - **Transizione:** NEW / HOLD (Disco) ⟶ READY (RAM).
> - **Collocazione:** Interfaccia tra memoria secondaria e RAM.
> - **Funzione Cardine:** Valuta se la RAM dispone di spazio sufficiente per ospitare un nuovo processo. Quando l'esito è positivo, distrugge il 🔵 **JCB** su disco e alloca il nuovo 🔵 **PCB** nella Ready Queue.
> - **Scopo Sistemistico:** Regola il grado complessivo di multiprogrammazione per non sovraccaricare la macchina.

💿 2. Medium-Term Scheduler (MTS / Swapper)

- **Transizione:** READY / WAITING (RAM) ⟷ WAITING ON DISK (Swap).
- **Collocazione:** Supervisore dello spazio di swapping su disco.
- **Funzione Cardine:** Quando la RAM fisica si satura (*Memory Pressure*), preleva processi inattivi e li sposta nell'area di swap su disco (**Swap-Out**). Non appena la memoria si libera, esegue lo **Swap-In** reinserendo il PCB nella Ready Queue.

---

2. Moduli di Esecuzione e Assegnazione CPU (RAM → CPU)

⚡ 3. Short-Term Scheduler (CPU / Task Scheduler / STS)

- **Transizione:** READY ⟶ RUNNING.
- **Collocazione:** Residente permanente in RAM e attivo a frequenza altissima (ogni 10 – 50 ms).
- **Funzione Cardine:** È il "cervello decisionale": analizza i processi pronti e seleziona il prossimo processo a cui concedere la CPU applicando l'algoritmo stabilito (Round-Robin, SJF, Priorità).

🚀 4. Dispatcher (Il Braccio Operativo del Kernel)

Riceve la scelta dello Short-Term Scheduler e realizza materialmente il passaggio della CPU compiendo **3 azioni critiche e delicate**:

1. 🟣 **Context Switch (Cambio di Contesto):** Salva nei registri del PCB uscente lo stato della CPU e ripristina i registri del nuovo PCB selezionato.
2. 🟢 **Commutazione di Privilegio:** Porta la CPU da Kernel Mode a User Mode.
3. 🟣 **Salto Operativo (Jump):** Carica l'indirizzo memorizzato nel Program Counter (PC) del nuovo processo per dare avvio all'esecuzione.

---

3. Moduli di Monitoraggio e Gestione Eventi Asincroni

🚦 5. Traffic Controller (Il Vigile del Traffico dei PCB)

- **Funzione Cardine:** Gestore e contabile delle strutture dati del Kernel.
- **Compiti Operativi:** Aggiorna il campo `Stato` all'interno di ciascun PCB e sposta fisicamente i puntatori delle liste concatenate in RAM (spostamento tra Ready Queue, Waiting Queue e lista dei Terminated).

⚡ 6. Interrupt Handler (Gestore delle Interruzioni Elettriche)

- **Transizione:** WAITING ⟶ READY.
- **Funzione Cardine:** Routine di bassissimo livello invocata istantaneamente quando arriva un segnale hardware (**IRQ**) da una periferica (tastiera, disco, scheda di rete).
- **Compiti Operativi:** Legge i dati dal buffer della periferica e segnala al SO che l'evento atteso si è verificato, rimettendo il processo in coda tra i pronti.

⚠️ LA REGOLA D'ORO DEL SISTEMA OPERATIVO (Domanda Trabocchetto da Esame!)

Un processo non può MAI passare direttamente dallo stato WAITING allo stato RUNNING!

Quando la periferica I/O termina il trasferimento o l'evento atteso si verifica, l'Interrupt Handler **sposta obbligatoriamente il processo nello stato di READY**.

La Motivazione Sistemistica (per la Prof): Nel momento esatto in cui la periferica finisce l'I/O, **la CPU è già fisicamente occupata da un altro processo in esecuzione**. L'hardware non può "estromettere" arbitrariamente il processo in esecuzione scavalcando lo Scheduler: ciò distruggerebbe ogni equità (*Fairness*) e violerebbe le politiche del sistema. Il processo sbloccato deve perciò mettersi in coda tra i pronti (*READY*), lasciando al CPU Scheduler la decisione di quando riassegnargli il processore.

3.2 Politiche e Algoritmi di Scheduling

Gli algoritmi di schedulazione rappresentano la logica con cui il CPU Scheduler decide a quale processo in READY assegnare la CPU. Si dividono in due grandi famiglie:

Algoritmi Non-Preemptive (Senza Prelazione)

Un algoritmo è **Non-Preemptive** se, una volta assegnata la CPU a un processo, questo **la mantiene ininterrottamente fino alla sua conclusione naturale o fino a quando non richiede spontaneamente un'operazione di I/O**. Il SO non interviene mai per sottrargli forzatamente il processore.

1. FCFS (First-Come, First-Served)

- **Logica:** I processi vengono serviti rigorosamente nell'ordine cronologico di arrivo nella Ready Queue (TA, politica FIFO - *First In, First Out*).
- **Vantaggi:** Semplicità assoluta di implementazione, overhead quasi nullo.
- **Svantaggio Critico:** 🔴 **Effetto Convoglio (Convoy Effect)**.

> 💡 **Che cos'è l'Effetto Convoglio in FCFS? (Motivazione per la Prof)**
>
> Immaginiamo una fila a un unico sportello postale in cui la prima persona deve svolgere un'operazione contabile lunghissima che richiede 60 minuti, mentre dietro di lei ci sono dieci persone che devono solo imbucare una cartolina da 10 secondi. In FCFS accade lo stesso: se entra per primo un processo lungo ad alta intensità di calcolo (**CPU-bound** con burst da 100 secondi), tutti i processi interattivi brevissimi (**I/O-bound** con burst da 0,1 secondi) rimangono intrappolati dietro di lui in attesa, facendo schizzare il tempo medio di attesa (T_{AM}) a valori inaccettabili.

2. SJF (Shortest Job First - Non-Preemptive)

- **Logica:** Quando la CPU si libera, lo scheduler scorre la Ready Queue e seleziona il processo che possiede il 🟠 **minor tempo di burst (T_B)** stimato.
- **Vantaggio Matematico:** È dimostrato matematicamente che l'algoritmo SJF è **ottimo**, poiché garantisce in assoluto il **minimo Tempo di Attesa Medio (T_{AM})** rispetto a qualsiasi altro

algoritmo non-preemptive.

- **I Due Limiti Fatali di SJF:**
  1. 🔴 **Rischio di Starvation (Inedia):** Se nel sistema continuano ad arrivare processi con burst breve, un processo con burst lungo rimarrà perennemente in fondo alla coda e non verrà **mai eseguito**.
  2. 🔴 **Non conoscibilità a priori del tempo di burst (T_B):** Nella realtà dei calcolatori moderni, il sistema operativo non può sapere in anticipo quanti millisecondi di calcolo richiederà un programma utente prima di fare I/O; il valore può essere soltanto stimato tramite medie esponenziali delle esecuzioni passate.

Algoritmi Preemptive (Con Prelazione)

Un algoritmo è **Preemptive** quando il Sistema Operativo possiede l'autorità hardware di **interrompere forzatamente il processo attualmente in RUNNING** (sfruttando il timer o i segnali di interrupt), togliergli la CPU, retrocederlo in READY e assegnare il processore a un altro processo più urgente o prioritario.

1. SRTF (Shortest Remaining Time First)

- **Logica:** È la versione **preemptive** dell'algoritmo SJF.
- Quando un nuovo processo arriva nella Ready Queue, lo scheduler confronta il suo burst time con il **tempo residuo rimanente** del processo attualmente in esecuzione sulla CPU:
  - Se il nuovo arrivato richiede meno tempo di quello che manca al processo corrente, **avviene la prelazione immediata**: il processo in RUNNING viene sospeso e la CPU viene data al nuovo arrivato.
  - A parità di tempo residuo, **non si fa preemption** per evitare l'overhead inutile di un context switch.

2. Round-Robin (RR)

- **Logica:** È l'algoritmo cardine dei sistemi Time-Sharing. I processi sono gestiti in una coda circolare FIFO. A ciascun processo viene concessa la CPU per un intervallo temporale prefissato pari a un 🟠 **Time-Slice (Quanto di Tempo)**.
- Se il processo termina o fa I/O prima dello scadere del time-slice, rilascia la CPU spontaneamente.
- Se scade il time-slice prima del completamento, un interrupt hardware scollega il processo e lo sposta in coda alla Ready Queue.

- **Punti di Forza:** 🟢 **Massima Equità (Fairness)**, eccellente tempo di risposta per programmi interattivi, e **assenza totale di starvation** (ogni processo ha la certezza matematica di avanzare a ogni giro di ruota).

---

4. Scheduling con Priorità e il Rimedio dell'Aging

Negli algoritmi a priorità, a ogni processo viene associato un valore numerico che ne determina l'urgenza di esecuzione:

> ⚖️ **Tipologie di Priorità nello Scheduling**
>
> - **Priorità Statica:** Assegnata alla nascita del processo e immutabile nel tempo.
>   🔴 **Pericolo Starvation:** se giungono continuamente task a priorità più alta, i processi a bassa priorità rimangono affamati all'infinito.
> - **Priorità Dinamica con AGING (Invecchiamento):** Il valore di priorità viene ricalcolato dal Kernel e incrementato progressivamente man mano che il processo attende nella Ready Queue.
>   🟢 **Garanzia di Esecuzione:** previene l'inedia e assicura che qualunque task prima o poi ottenga la CPU.

- **Il Concetto di Niceness in Linux/Unix:** Nei sistemi Unix e Linux, la priorità dei processi nello spazio utente è regolata dal valore di 🔵 **Niceness (Gentilezza)**, compreso tra **−20 e +19**:
  - Un valore di niceness pari a **−20** indica priorità **massima** (il processo è "egoista", non è gentile con gli altri e pretende tutta la CPU).
  - Un valore di niceness pari a **+19** indica priorità **minima** (il processo è "molto gentile", cede il passo a chiunque).

> 💡 **Come si risolve la Starvation causata dalle Priorità? (Motivazione per la Prof)**
>
> Se utilizzassimo solo priorità statiche, un processo a bassa priorità potrebbe rimanere bloccato in Ready Queue per giorni o settimane (**Starvation**), non appena il sistema riceve costantemente processi a priorità più alta. La soluzione ingegneristica standard si chiama 🟢 **Aging (Invecchiamento)**: Il Kernel monitora costantemente il tempo che ciascun processo trascorre fermo nella coda di READY. A intervalli regolari, **il SO incrementa**

> **gradualmente la priorità del processo affamato**. Con il passare del tempo, la sua priorità diventerà così elevata da superare anche i processi appena arrivati, garantendo l'esecuzione certa di qualunque elaborazione.

📊 Metriche di Prestazione e Caso Reale d'Esame

La valutazione quantitativa di un algoritmo di scheduling si basa su metriche matematiche precise. In questa sezione viene presentato il **formulario formale completo** e la risoluzione analitica guidata di un **tipico problema d'esame** caratterizzato da interruzioni hardware e prelazione della CPU.

4.1 Formulario Formale delle Metriche di Scheduling

> 📐 **Le 4 Metriche di Prestazione Cardine dello Scheduling**
>
> 1. 1️⃣ **Tempo di Turnaround (T Ti):** Tempo totale dal momento dell'ingresso nel sistema fino al completamento definitivo (Tfine – Tarrivo).
> 2. 2️⃣ **Tempo di Attesa (T Ai):** Tempo complessivo trascorso fermo in RAM o in I/O senza poter usare la CPU (T Ti – Tburst).
> 3. 3️⃣ **Efficienza della CPU (η):** Frazione percentuale del tempo totale spesa in calcolo utile per i processi utente (Ttotale∑ Tburst × 100%).
> 4. 4️⃣ **Throughput del Sistema:** Numero di processi portati a termine con successo per unità di tempo.

---

1. Tempo di Turnaround (T Ti - Tempo di Completamento)

> **Definizione di Turnaround Time**

> Il **Tempo di Turnaround (T Ti)** rappresenta l'intervallo temporale complessivo che intercorre tra l'istante in cui il processo entra per la prima volta nel sistema (stato *NEW/HOLD*) e l'istante in cui conclude definitivamente la propria esecuzione (stato *TERMINATED*):

TTi = Tfine,i – Tarrivo,i

Il **Tempo di Turnaround Medio (T TM)** su un insieme di N processi è la media aritmetica dei singoli tempi di turnaround: TTM = N∑i=1N TTi = NTT1+TT2+⋯+TTN

---

2. Tempo di Attesa (T Ai - Waiting Time)

> **Definizione di Tempo di Attesa**
>
> Il **Tempo di Attesa (T Ai)** è la somma di tutti gli intervalli temporali in cui un processo è rimasto inattivo, in coda nella memoria RAM (stato *READY*) o in attesa del completamento di un'operazione di I/O (stato *WAITING*), senza poter utilizzare la CPU:

TAi = TTi – Tburst,i

Per i processi che subiscono preemption (interruzioni multiple), il tempo di attesa può essere calcolato anche sommando direttamente i singoli segmenti temporali trascorsi in attesa lungo l'asse dei tempi: TA(P) = (Tinizio1 – Tarrivo) + (Tripresa1 – Tsospensione1) + ...

Il **Tempo di Attesa Medio (T AM)** su N processi è dato da: TAM = N∑i=1N TAi = NTA1+TA2+⋯ +TAN

---

3. Efficienza della CPU (η)

> **Definizione di Efficienza della CPU**
>
> L'**Efficienza della CPU (η)** rappresenta la frazione percentuale del tempo di osservazione in cui la CPU è stata attivamente impegnata nell'elaborazione del codice utile dei programmi

utente:

$\eta = T_{totale} \sum_{i=1}^{N} T_{burst,i} \times 100\%$

Scomponendo il tempo totale di osservazione del sistema nelle sue componenti fisiche reali:
$T_{totale} = \sum_{i=1}^{N} T_{burst,i} + T_{idle} + T_{overhead}$

Dove:

- $\sum T_{burst,i}$: somma dei tempi di esecuzione utile dei processi.
- 🔴 **T_{idle}:** tempo in cui la CPU è rimasta completamente inattiva per assenza di processi pronti.
- 🔴 **T_{overhead}:** tempo speso dal sistema operativo per eseguire context switch, aggiornamento dei PCB e routine del kernel.

> 💡 **Perché nei sistemi reali risulta sempre $\eta < 100\%$? (Motivazione per la Prof)**
>
> È fisicamente impossibile raggiungere un'efficienza del 100% perché il passaggio da un processo all'altro non è istantaneo: richiede il tempo di cambio di contesto del Dispatcher ($T_{overhead}$). Inoltre, possono verificarsi finestre temporali in cui nessun processo è pronto in RAM (T_{idle}), durante le quali la CPU esegue unicamente il ciclo nullo a basso consumo (*HLT / idle loop*).

---

4. Throughput (Produttività del Sistema)

> **Definizione di Throughput**
>
> Il **Throughput** misura il numero di processi che il sistema è in grado di completare con successo all'interno di una determinata unità di tempo (es. processi al secondo o processi per minuto):

Throughput = Intervallo di Tempo OsservatoNumero di Processi Completati

---

4.2 Analisi Completa del Caso Reale d'Esame: Interruzione Hardware

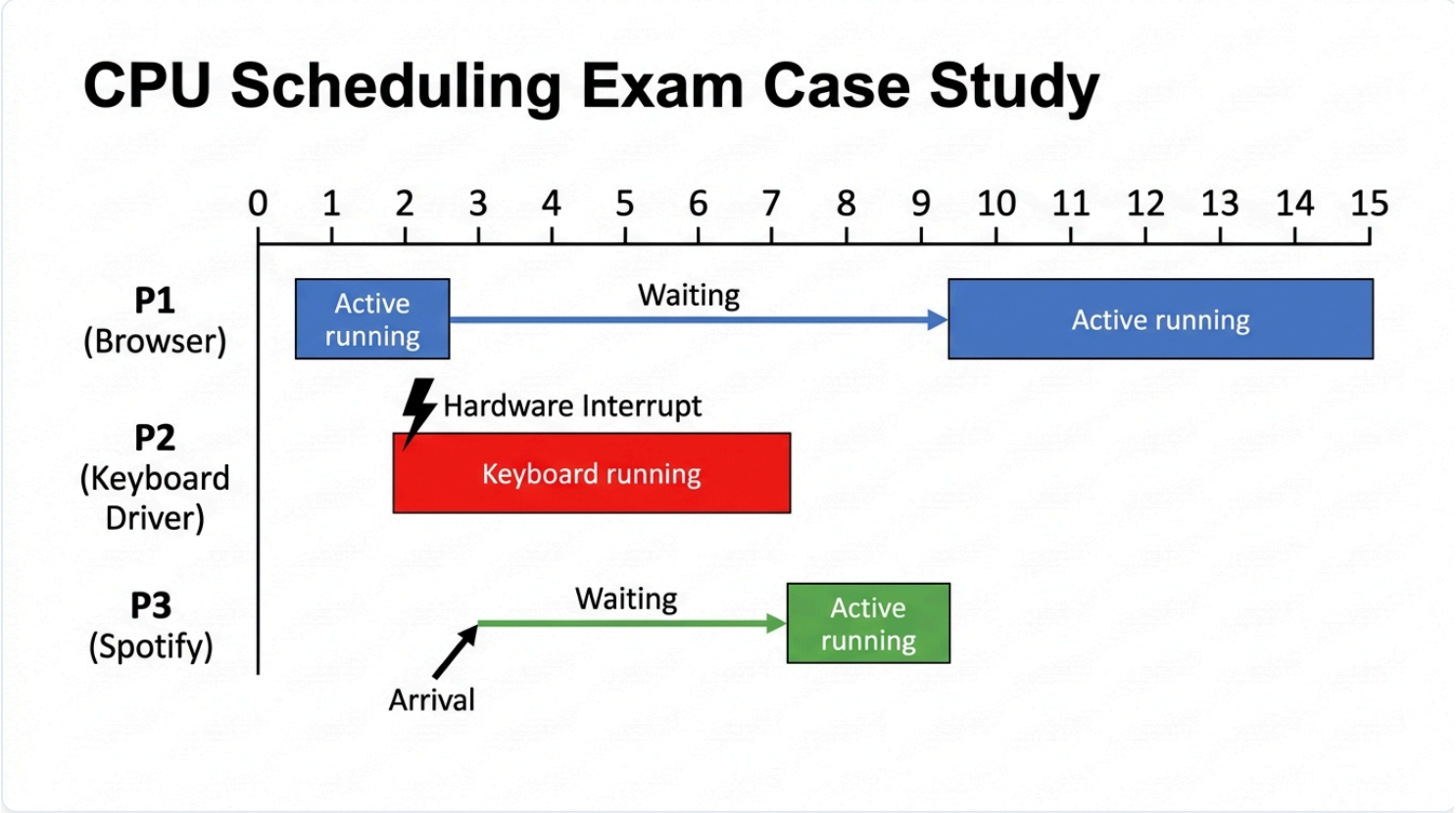
Questa tipologia di esercizio è tra le più gettonate nei compiti in classe e negli esami di TPSIT, poiché verifica la capacità dello studente di comprendere le **interruzioni hardware**, la **prelazione** e il calcolo rigoroso delle metriche.

Traccia dello Scenario

All'interno del sistema operano **3 processi distinti**:

- 🔵 **Processo P_1 (Browser di navigazione):** processo utente a priorità standard.
  - Istante di arrivo: TA = 1
  - Tempo di burst richiesto: TB = 7
- 🔴 **Processo P_2 (Driver della tastiera):** processo di sistema collegato a una periferica fisica ad alta priorità hardware.
  - Istante di arrivo (evento tasto premuto): TA = 2
  - Tempo di burst richiesto: TB = 5
- 🟢 **Processo P_3 (Riproduttore audio Spotify):** processo utente standard.
  - Istante di arrivo: TA = 3
  - Tempo di burst richiesto: TB = 2

Diagramma Temporale di Gantt (Risoluzione Visiva)



📊 Legenda Visiva del Diagramma di Gantt

- **T = 0 → T = 1:** 🟠 **IDLE** (CPU in attesa).
- **T = 1 → T = 2:** 🔵 **P_1 RUNNING** (esegue 1 unità).
- **T = 2:** ⚡ **Interruzione Hardware (IRQ)**: P2 fa prelazione su P1. P1 torna in 🟢 **READY** con burst residuo pari a 6.
- **T = 2 → T = 7:** 🔴 **P_2 RUNNING** (esegue 5 unità non interrompibili da processi utente). A T = 3 arriva P3 e si mette in coda 🟢 **READY**. A T = 7, P2 termina (🔴 **TERMINATED**).
- **T = 7 → T = 9:** 🟢 **P_3 RUNNING** (SJF: burst 2 < residuo 6). A T = 9, P3 termina.
- **T = 9 → T = 15:** 🔵 **P_1 RUNNING** (completa le 6 unità residue e termina a T = 15).

Cronologia Analitica Istante per Istante con le Motivazioni per la Prof

⏱️ Istante T = 0:

- Nel sistema non è ancora giunto alcun processo.

- La CPU si trova nello stato 🔴 **IDLE (Inattività)** per la durata di 1 unità di tempo.

⏱️ Istante T = 1:

- Arriva il processo P1 ($TA = 1, TB = 7$).
- Poiché la CPU è libera, il CPU Scheduler e il Dispatcher allocano immediatamente il processore a P1.
- **Stato:** P1 entra in ⚡ **RUNNING**.

⏱️ Istante T = 2 (L'Evento Chiave: Interruzione Hardware):

- L'utente preme fisicamente un tasto sulla tastiera.
- Il controller della tastiera invia una linea di interrupt elettrica (**IRQ**) alla CPU.
- L'Interrupt Handler risveglia il driver P2 ($TA = 2, TB = 5$).
- **La Decisione di Scheduling (per la Prof):** P2 è un driver di gestione hardware critico del sistema operativo e gode di **priorità hardware assoluta** rispetto al browser utente P1. Di conseguenza, la CPU attiva la 🔴 **prelazione forzata (Preemption)**:
  1. P1 viene sospeso e retrocesso forzatamente nello stato 🟢 **READY**.
  2. P1 ha consumato 1 sola unità di calcolo (da $T = 1$ a $T = 2$), quindi il suo **burst residuo rimanente è pari a 7 – 1 = 6 unità**.
  3. P2 viene caricato nei registri della CPU ed entra in ⚡ **RUNNING**.

⏱️ Istante T = 3:

- Entra nel sistema il processo P3 (Spotify, con $TA = 3$ e $TB = 2$).
- **La Decisione di Scheduling (per la Prof):** Nonostante P3 sia un processo brevissimo ($TB = 2$) e in una normale politica SJF avrebbe la precedenza su burst più lunghi, **la routine del driver hardware P2 in esecuzione non può essere interrotta da normali processi utente**. P3 deve pertanto accodarsi pazientemente nella coda di 🟢 **READY** in memoria RAM.

⏱️ Istante T = 7:

- Il driver P2 completa l'intero burst di 5 unità ($2 + 5 = 7$) e transita nello stato 🔴 **TERMINATED** ($TF\,2 = 7$).
- La CPU torna disponibile. Nella Ready Queue sono presenti due processi in attesa:
  - P3 con burst $TB = 2$
  - P1 con burst residuo $TB,residuo = 6$

- **La Decisione di Scheduling (per la Prof):** Trattandosi di due normali processi a livello utente, lo Scheduler applica la politica **SJF (Shortest Job First)** per minimizzare il tempo di attesa medio. Poiché il tempo di P3 (2) è nettamente inferiore al tempo di P1 (6), la CPU viene assegnata a **P3**.
- P3 transita in ⚡ **RUNNING** per 2 unità di tempo (da $T = 7$ a $T = 9$).

⏱️ Istante $T = 9$:

- P3 conclude regolarmente il proprio burst e termina ($TF_3 = 9$).
- L'unico processo rimasto nella Ready Queue è P1.
- Il Dispatcher esegue il Context Switch, ripristina i registri di P1 dal PCB e gli riassegna la CPU.
- P1 esegue le sue rimanenti 6 unità di burst (da $T = 9$ a $T = 15$).

⏱️ Istante $T = 15$:

- P1 termina definitivamente la propria esecuzione ($TF_1 = 15$).

4.3 Calcoli Matematici Formali del Problema

Raccogliamo tutti i valori all'interno della tabella analitica:

Processo	Descrizione	Istante Arrivo (TA)	Burst Totale (TB)	Istante Fine (Tfine)	Turnaround (TT = Tfine – TA)	Attesa (TA = TT – TB)
P1	Browser Web (Utente)	1	7	**15**	15 – 1 = 14	14 – 7 = 7
P2	Driver Tastiera (HW)	2	5	**7**	7 – 2 = 5	5 – 5 = 0
P3	Riproduttore Audio	3	2	**9**	9 – 3 = 6	6 – 2 = 4

---

1. Calcolo dei Tempi di Turnaround

- TT1 = 15 – 1 = 14
- TT2 = 7 – 2 = 5
- TT3 = 9 – 3 = 6

TTM = 314+5+6 = 325 ≈ 8,33 unita` di tempo

---

2. Calcolo dei Tempi di Attesa

- **Verifica diretta del tempo di attesa di P1 sull'asse temporale:**
  - P1 non attende prima di iniziare (T = 1).
  - Viene sospeso all'istante T = 2 e riprende all'istante T = 9.
  - Rimane fermo in READY da T = 2 a T = 9, ovvero per 9 – 2 = 7 unita` di tempo (esattamente la durata dell'esecuzione combinata di P2 e P3).
- **Verifica del tempo di attesa di P2:**
  - Arriva a T = 2 e parte immediatamente a T = 2 $\implies$ TA2 = 0.
- **Verifica del tempo di attesa di P3:**
  - Arriva a T = 3 e rimane bloccato in READY fino a T = 7 $\implies$ 7 – 3 = 4 unita` di tempo.

TAM = 37+0+4 = 311 ≈ 3,67 unita` di tempo

---

3. Calcolo dell'Efficienza della CPU (η)

- Somma dei tempi di burst utili eseguiti dalla CPU: ∑TB = TB1 + TB2 + TB3 = 7 + 5 + 2 = 14 unita`
- Tempo totale trascorso di osservazione del sistema: Ttotale = 15 unita`
- Tempo di inattività della CPU (*Idle Time*): Tidle = 1 unita` (da T = 0 a T = 1)

η = Ttotale∑TB × 100% = 1514 × 100% ≈ 93,33%

> **Riscontro dell'Efficienza**
>
> La CPU ha lavorato per il 93,33% del tempo a disposizione. L'unico momento di inefficienza è stato l'intervallo iniziale da T = 0 a T = 1 (6,67%), in cui nessun processo era ancora arrivato nel sistema.